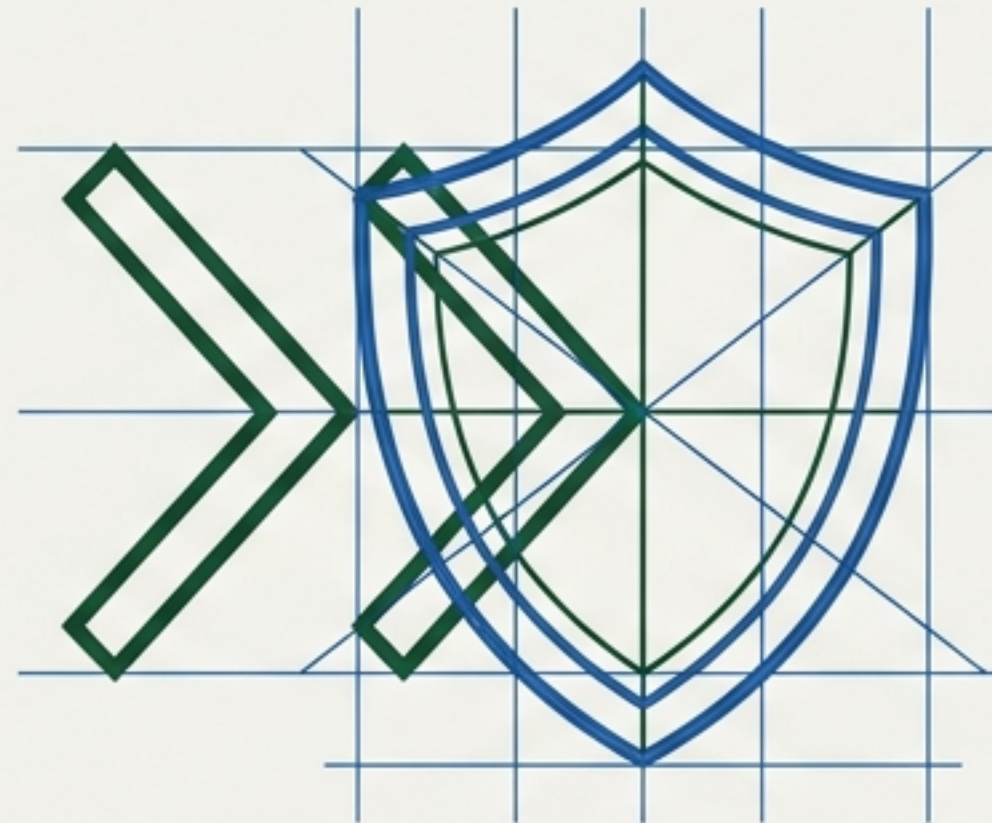


MongoDB Basics & CRUD

The Viva Survival Guide

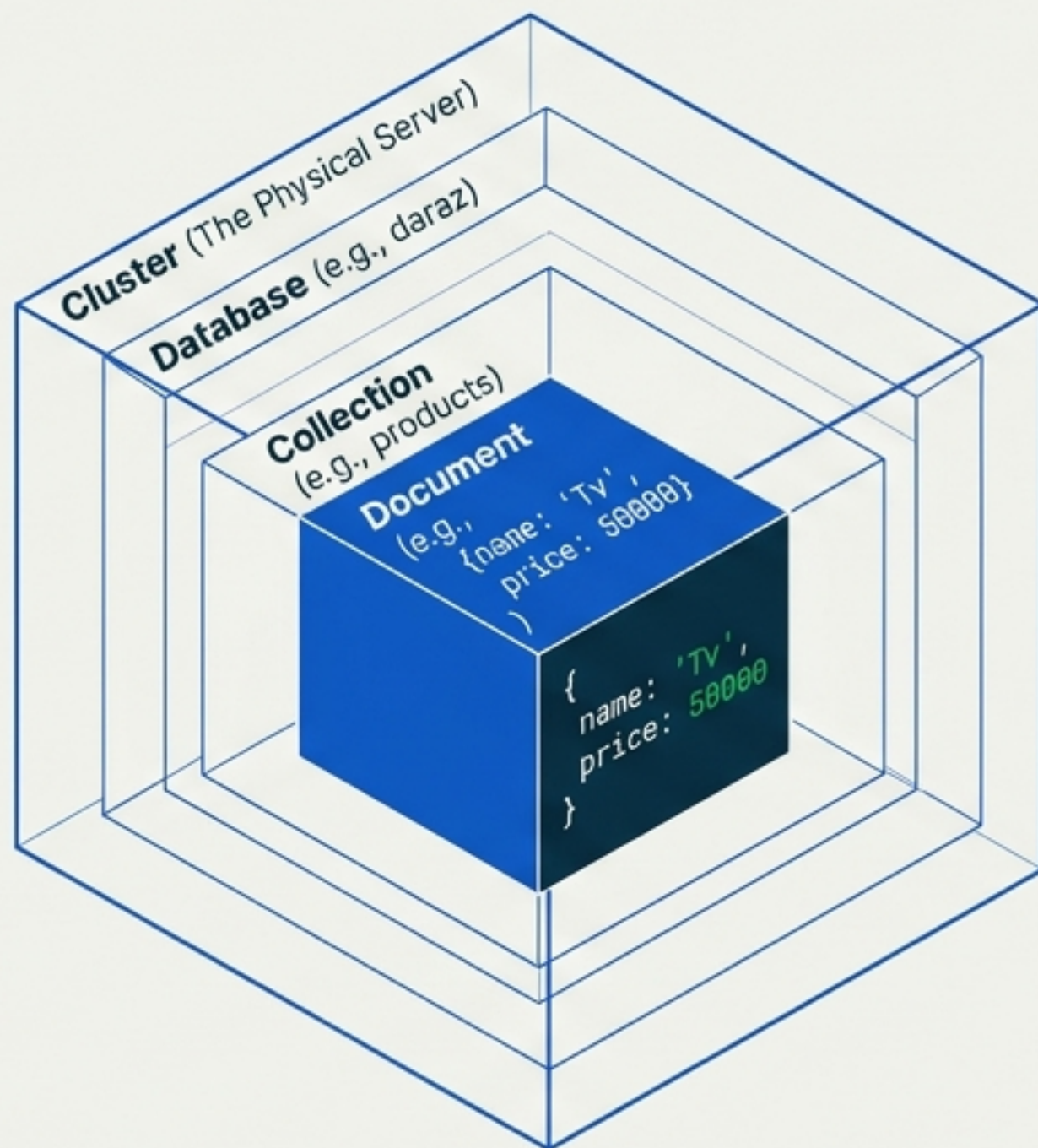
- Tactical frameworks, mnemonics, and examiner traps to master NoSQL data operations under pressure.



[SYSTEM: MONGOSH // TARGET: EXAM PREP // STATUS: READY]

The Architectural Hierarchy of MongoDB

**Mnemonic:
C-D-C-D**



MongoDB is schema-less, but strictly hierarchical.

Never lose your place in the tree.

Terminal Navigation and Naming Rules

The Commands

`mongosh`

→ Enter terminal

`show dbs`

→ List databases

`use <databaseName>`

→ Select database
(e.g., `use daraz`)

`db.createCollection('<name>')`

→ Explicit creation

`show collections`

→ List collections

The Law: Naming Conventions

Database Name = Singular (e.g., daraz, app)

Collection Name = Plural (e.g., products, users)

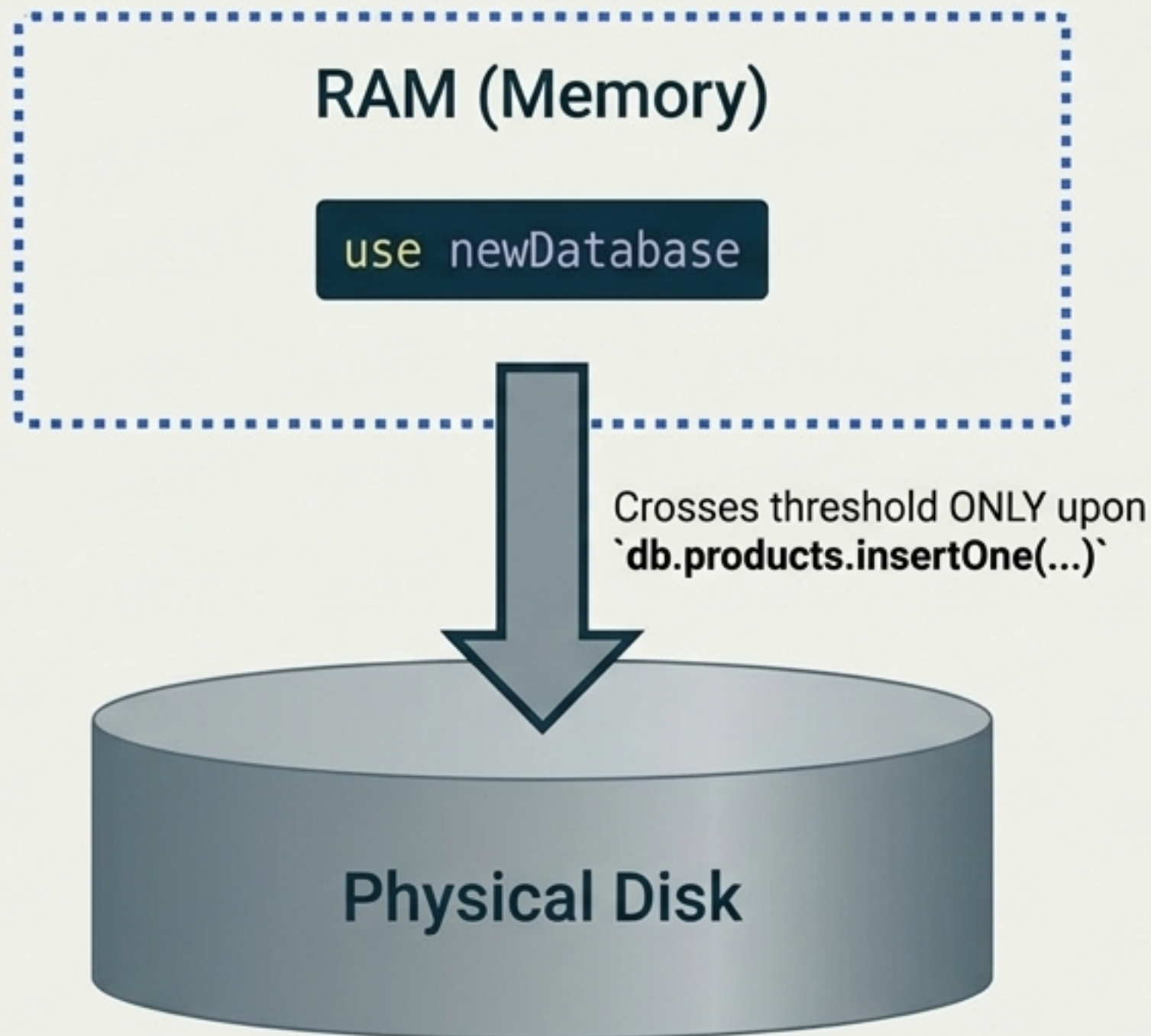


Examiner Check: Examiners check collection names to see if you respect industry standards.

Examiner Trap: The Lazy Creation Principle

The Trap

What happens if you try to use a database that doesn't exist?



The Reality

MongoDB switches to it in memory immediately, but the database and collection are not actually created on disk until the first document is inserted.

The Operational Matrix

Mnemonic: **CRUD**

Create

Add documents.

``insertOne`, `insertMany``

Read

Retrieve documents.

``find``

Update

Modify existing documents.

``updateOne`, `updateMany``

Delete

Remove documents.

``deleteOne`, `deleteMany``

Four verbs rule the data layer. Every complex query is just a variation of these base operations.

Injecting Data and Implicit Collections

```
db.products.insertMany([ {name: 'Tv', price: 50000},  
                          {name: 'Mobile', price: 20000} ])
```

○ Target
current
database

○ Target collection
(creates it if it doesn't exist)

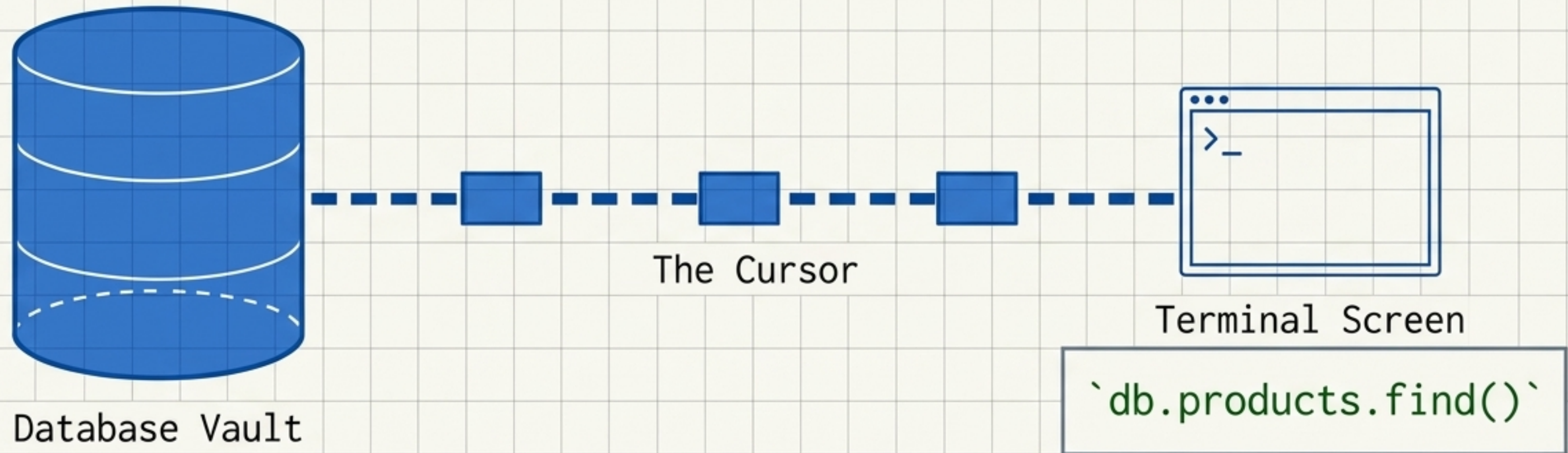
○ The operational verb

○ The array of BSON
documents

Examiner Note:

If `products` didn't exist before this command, MongoDB just built it for you. This is schema-less flexibility in action.

Extracting Data via the Find Cursor



MongoDB does not dump all raw data onto your screen at once. ``find()`` returns a cursor—a pointer to the result set that fetches data in batches to preserve memory.

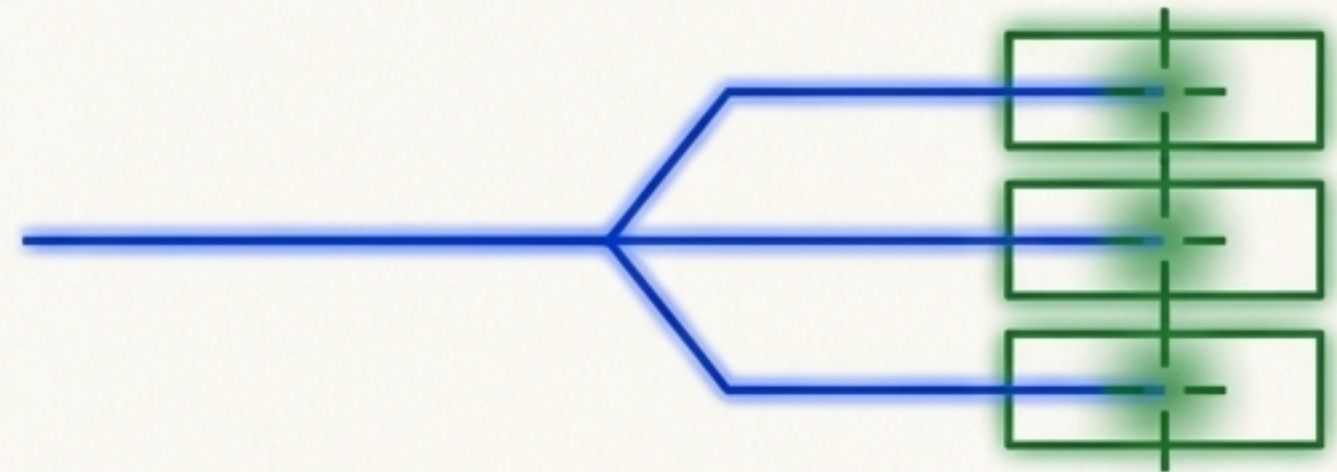
The First Match Trap and Update Scopes

updateOne



```
db.products.updateOne({name: 'Tv'},  
{$set:{price:120000}})
```

updateMany



```
db.products.updateMany({name: 'Tv'},  
{$set:{price:120000}})
```

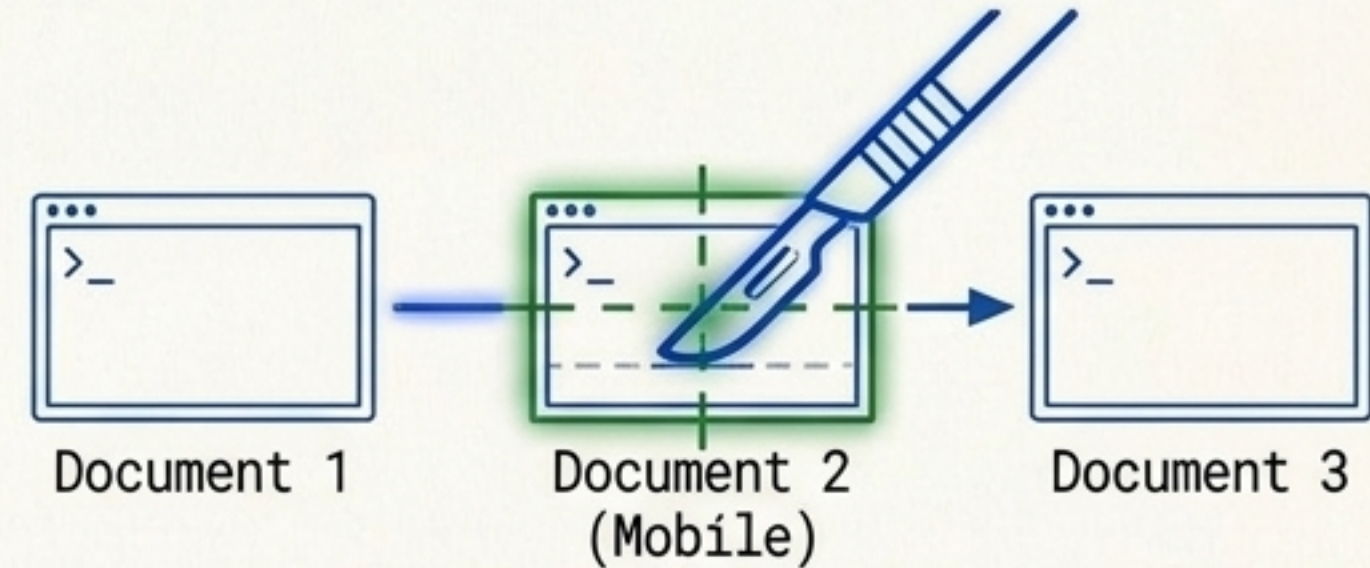
Examiner Note:

The **\$set** operator is non-negotiable. Forgetting it will overwrite the entire document with just the updated field. Always wrap modifications in **\$set**.

Eradicating Documents (With Caution)

Safe Execution

```
db.products.deleteOne({name:'Mobile'})
```



The Catastrophe Trap

```
db.products.deleteMany({})
```



Examiner Note:

An empty filter object `{}` in a `deleteMany` command will wipe the entire collection. Examiners will ask how to clear a collection without dropping it—this is the answer, but warn them of the danger.

Architectural Synthesis: Native Shell vs. Mongoose ORM

Native MongoDB (Database Shell)	Mongoose ORM (Node.js Application)
<code>db.products.insertOne(data)</code>	<code>Product.create(data)</code>
<code>db.products.find()</code>	<code>Product.find()</code>
<code>db.products.updateOne(filter, data)</code>	<code>Product.findByIdAndUpdate(id, data)</code>
<code>db.products.deleteOne(filter)</code>	<code>Product.findByIdAndDelete(id)</code>

Native commands execute directly in the database shell. Mongoose methods run in your Node.js application, adding validation and schema enforcement.

Viva Interrogation Simulator

Scenario 01

Q: What happens if you run ``use secretDB but never insert data?`

A: The DB exists only in **memory (RAM)**. It is not written to the physical disk until the first document is inserted (**Lazy Creation**).

Scenario 02

Q: There are 10 'Tv' documents. You run `updateOne({name: 'Tv'}, {$set: {price: 0}}).` What happens?

A: Only the **FIRST 'Tv' document** encountered in the collection gets a price of 0. The other 9 remain **untouched**.

The Tactical Dossier (Master Cheat Sheet)

Mnemonic: C-D-C-D (Cluster > DB > Collection > Document)

Shell Commands

- `mongosh`
- `show dbs`
- `use <db>`
- `show collections`

CRUD Syntax

- `insertOne`
- `find`
- `updateOne` + `$set`
- `deleteOne`

Examiner Traps

- **Lazy Disk** (No write without insert)
- **First Match Rule** (`updateOne` hits only one)
- **Empty Filter** (`deleteMany({})` wipes all)
- **Plural Collections / Singular DBs**